

Checking Failure-Path Telemetry in the Data a Project Already Emits

Halley Young
Preprint, October 2026

Abstract—When a request fails in production, responders need the emitted telemetry to say which request failed and why. We present **TELEMETRY-CONTRACTS**, a pure-Python command-line tool that flags missing recognized fields associated with this property in the logs, traces and metrics a project already has, with no contract written in advance and no instrumentation convention to adopt. It defines six gap classes with written definitions and checks each one as a deterministic function of the events. On a 112-item gold set the tool reaches F1 0.953 and outperforms two rule baselines (exact McNemar $p < 0.001$). Two zero-shot LLM judges given the same class definitions reach 0.991 (Claude Haiku 4.5) and 0.952 (GPT-4.1-mini). The tool and the judges disagree on only four and six items, and at that sample size neither difference is significant ($p = 0.125$ and $p = 1.00$). The tool’s F1 falls between the two judges’ while it runs offline and deterministically at no per-event cost. We ran it over 58 pinned public repositories (116,420 events, 2,269 findings). In an LLM-labelled stratified sample of 60 findings, the two failure-path checks matched their definitions in 25 of 30 cases. The tool, gold set, corpus manifest, prompts, raw model outputs, validation labels and a blinded annotation packet for independent raters are released.

I. INTRODUCTION

Logging research has long studied whether code logs enough to diagnose failures. It has measured logging practice [1], [2], [3], added diagnostic variables to existing log statements [4], added log statements on error paths [5], and predicted where developers should log [6], [7], [8]. These approaches act on source code. They answer the question “should this code location log?”

This paper asks a narrower question about the output. Given the telemetry a service has emitted, does each failure carry a recognized correlation identifier, and does it carry a recognised field for error or status evidence? Distributed tracing makes the first property routine in principle [9]. In practice teams emit heterogeneous JSON logs, logfmt lines and OTLP exports, under field names that follow no single convention. The OpenTelemetry semantic conventions [10] standardise names for the teams that adopt them.

TELEMETRY-CONTRACTS treats each such property as a contract clause over a finite set of events and reports violations. The checks run on whatever a project ships, so they apply to repositories whose authors never used the tool. Our claim is that missing fields for the two failure-path checks can be detected on heterogeneous telemetry at the cost and determinism of a linter, with gold-set accuracy in the range of two zero-shot LLM judges. The contributions are:

- six gap classes with written definitions, and a checker that reads JSONL, JSON arrays, logfmt and OTLP exports without configuration (Sections III and IV);
- a gold-set comparison against two rule baselines and two zero-shot LLMs, with every prompt and raw response released;
- a study over 58 pinned public repositories with an LLM-labelled stratified sample of 60 findings and a released annotation protocol.

We address three questions:

- **RQ1.** How accurate are the checks on a labelled set of telemetry samples?
- **RQ2.** How do they compare with rule baselines and with zero-shot LLMs given the same definitions?
- **RQ3.** When run on public repositories, how often do the findings match their definitions?

II. RELATED WORK

Logging practice and log enhancement. Empirical studies characterise how and where developers log in C/C++, Java, industrial and mobile code [1], [2], [6], [3], and what logging costs and buys [11]. LogEnhancer adds causally relevant variables to existing log statements [4]. Errlog inserts log statements at exception and error sites [5]. Learning-based approaches recommend where to log [7], [8], and LLM-based approaches generate the statements themselves [12], [13]. Chen and Jiang detect anti-patterns in logging code [14]. All of these analyse or change source code. **TELEMETRY-CONTRACTS** examines the emitted output instead, so it checks repositories in any language without building them. SherLog combines source code with run-time logs to infer control and data facts about a failed execution [15]. Its diagnostic target is stronger than field coverage. We have not compared incident diagnosis against SherLog or LogEnhancer; the present evidence concerns missing recognized fields.

Log analysis. Automated log analysis parses, mines and detects anomalies in logs once they exist [16], [17], and Loghub provides datasets for that work [18]. These techniques take the logs as given. **TELEMETRY-CONTRACTS** checks whether the logs can answer specific incident questions.

Specification mining and runtime verification. Specification miners infer temporal or invariant specifications from traces [19], [20], [21]. Runtime verification checks traces against temporal-logic specifications [22], [23], and hyper-properties capture information-flow requirements over sets of traces [24]. **TELEMETRY-CONTRACTS**’s contract inference and

temporal clauses build on these ideas. The checks evaluated here are per-event properties.

Conventions and scanners. The OpenTelemetry semantic conventions [10] define field names, and our `semconv-only` baseline approximates a conformance linter. Secret scanners such as `detect-secrets` and `gitleaks` look for credentials in source files. The sensitive-value check applies similar patterns to telemetry values.

III. GAP CLASSES AND CHECKS

An *event* is a JSON object after format normalisation (Section IV). A *failure event* has severity `error`, `critical` or `fatal`, or a `status/outcome` field whose value denotes failure. The checks are:

- **missing-correlation:** A failure event with no correlation identifier (`trace_id`, `traceId`, `request_id`, `span_id` or a recognised alias).
- **missing-error-evidence:** A failure event with no error code, exception type or status field.
- **missing-duration:** A span event with no duration or latency measure.
- **sensitive-values:** A scalar value matching a raw-secret or PII pattern, namely an email, bearer token, JWT, password assignment, or a card or phone number in a field with a card or phone name.
- **unclassified-sensitive:** A field whose name denotes a tenant, customer, account or user identifier, or another sensitive attribute, emitted without redaction, hashing or a privacy classification.
- **unbounded-cardinality:** A metric label whose distinct-value count is high relative to the sample, reported only above a minimum sample size.

The first two checks require a nonblank scalar under a recognised field name. They reject container, boolean and whitespace placeholders. A status field can describe failure without identifying its cause. The duration check recognises numeric measures. These checks establish gaps in a field projection, not that a request is unidentifiable or a root cause unrecoverable from other evidence. The last three are pattern or statistical heuristics tuned for precision. Every finding records its check code, the index of the offending event, a remediation hint and, in directory and repository scans, the source file. The `sarif` command converts a JSON findings report to SARIF for code-scanning pipelines.

IV. IMPLEMENTATION

`TELEMETRY-CONTRACTS` is a standard-library-only Python package with a command-line interface and a GitHub Action. Adapters read JSONL, JSON arrays, logfmt and OTLP/JSON collector exports. They map common aliases (`level/severity`, `msg/message`, `service.name`, `traceId`) and infer the signal kind with a confidence score. The `scan-repo` command shallow-clones a repository (`git clone --depth 1`) at its default branch or at a branch or tag given with `--ref`, discovers candidate telemetry files with a bounded walk that skips configuration, lock files and

vendored directories, and checks each file. For each check code it keeps at most 25 example findings per file, so per-file counts are lower bounds. For studies, `corpus-download` checks out every repository in a manifest at its pinned commit SHA, and `corpus-run` scans the checkouts with a content-addressed result cache. Every aggregate uses integer or fixed-point arithmetic. The repository’s CI runs `reproduce --check`, which regenerates 13 artifacts, among them the gold-set and baseline reports, and requires each to match a committed SHA-256 manifest byte for byte.

V. EVALUATION

All numbers below are regenerated by the commands in `REPRODUCE.md`. Raw outputs are under `results/` and `reports/`.

A. RQ1: Gold-set accuracy

Setup. The gold set (`benchmarks/ground_truth/curated.jsonl`) has 112 (sample, class) pairs, 54 positive and 58 negative, covering all six classes. Each author-written label records the intended semantic gap, including evidence outside the detector’s field lists; these labels are not independent human judgments. Five *hard* items probe the edges of field-keyed checking, such as a correlation id under the key `correlationGuid` and a base64-encoded credential. The tool’s author wrote the labels.

Results. The tool scores precision 0.962 (51/53, Wilson 95% CI 0.87–0.99), recall 0.944 (51/54, 0.85–0.98) and F1 0.953. Per-class F1 ranges from 0.923 (unbounded-cardinality) to 1.000 (missing-duration). The five errors are the five hard items. Two are false positives, a correlation id under the unrecognised key `correlationGuid` and error evidence present only in a free-text message. Three are false negatives, a base64-encoded credential, a tenant identifier under the unrecognised name `org_id` and a cardinality sample smaller than the minimum the check requires before it reports. Two errors are field names outside the alias tables, one is an encoding the patterns do not decode, one is evidence in free text, which the presence checks do not read, and one is the sample-size gate.

B. RQ2: Rule baselines and LLMs

Setup. Every method receives only the events and the class name, never the label, and is scored by the same code. *rule-light* checks only for obvious field names. *semconv-only* flags missing OpenTelemetry convention fields, and its three classes without a convention field are reported out of scope. For the LLM rows we sent each item once through OpenRouter to `anthropic/claude-haiku-4.5` and `openai/gpt-4.1-mini`, at temperature 0 with at most 200 output tokens. The prompt states the class name, its definition and the events as JSON, and asks for YES or NO plus one sentence. All 224 responses parsed. The raw responses, served model ids, token counts and costs are in `results/llm_baseline/`. Both runs together cost

TABLE I

GOLD SET (N = 112). COVERED-CLASS F1 EXCLUDES CLASSES A METHOD CANNOT EXPRESS. DISCORDANT PAIRS: ITEMS ONLY THE TOOL GETS RIGHT / ITEMS ONLY THE OTHER METHOD GETS RIGHT.

Method	P	R	F1	Cov. F1	Disc. (p)
TELEMETRY-CONTRACTS	.962	.944	.953	.953	—
rule-light	.857	.667	.750	.809	20/1 (<.001)
semconv-only	.628	.500	.557	.771	38/0 (<.001)
Claude Haiku 4.5	.982	1.00	.991	.991	0/4 (.125)
GPT-4.1-mini	.980	.926	.952	.952	3/3 (1.00)

US\$0.077. The comparison replays the recorded verdicts of fine. Significance is an exact two-sided McNemar test on the paired per-item outcomes.

Results. Table I shows the results. The tool outperforms both rule baselines significantly. Claude Haiku 4.5 has the highest F1, 0.991 against the tool’s 0.953, and is right on all four items where the two disagree. These are the two keys outside the alias table (`correlationGuid`, `org_id`), the base64-encoded credential and the small cardinality sample. Four discordant items is too few for an exact McNemar test to reach significance, since the smallest attainable p is 0.125. GPT-4.1-mini reaches 0.952. It resolves three hard items (the correlation key, the base64 credential and the small sample) and misses three unclassified-identifier positives that the tool catches, a 3/3 split ($p = 1.00$). No significant difference from either judge is detected at this sample size. On the two failure-path classes (41 items) the tool makes two errors and each judge makes one. The fifth hard item, error evidence present only in free text, is labelled negative and every method flags it. The tool delivers this accuracy with a linter’s operating profile. It runs offline with no API key, gives the same output on every run, and makes no model calls. An LLM judge needs one call per (event sample, class) pair, about 300 prompt tokens per item on this set, which on a repository scan grows with the number of events.

C. RQ3: Corpus study with LLM-labelled findings

Setup. The corpus manifest (`benchmarks/corpus/corpus.json`) pins 59 public repositories at full commit SHAs. A curation script selected 54 of them. It searched GitHub code for committed telemetry files, cloned each candidate and kept those whose files parse as telemetry. The other five were chosen by hand as a small multi-host subset, three GitHub repositories that ship structured logs and two sample projects on GitLab and Bitbucket that ship no telemetry and serve as controls for a scan with nothing to report. Of the 59, 58 could be cloned (one curated repository was deleted upstream), 56 of them on GitHub. Of the 58, 34 are primarily Python, and the rest include Go, Rust, TypeScript and Java. With the repositories already downloaded, `corpus-run` scanned all 58 in 8.5 minutes on a laptop, excluding clone time. The archived scan contains 2,269 findings with redacted event excerpts and drew a stratified sample of 15 findings per class that fired, with at most two per (repository, class), ordered by a hash.

TABLE II

RQ3: PREVALENCE OVER 56 TELEMETRY-BEARING REPOSITORIES, AND LLM-LABELLED PRECISION ON 15 SAMPLED FINDINGS PER CLASS (WILSON 95% CI). FINDING COUNTS ARE CAPPED AT 25 PER CHECK PER FILE. BORDERLINE (SESSION-ID) CASES COUNT AS INCORRECT.

Class	Repos	Findings	Matches definition
missing-correlation	10	171	13/15 (.62–.96)
missing-error-ev.	10	127	12/15 (.55–.93)
sensitive-values	10	110	9/15 (.36–.80)
unclassified-sens.	19	1,861	1/15 (+9 borderline)
All sampled			35/60 (.46–.70)

Labelling procedure. An LLM agent inspected each sampled finding. It read the flagged event in the pinned check-out and the class definition, then assigned three labels. The first is whether the event matches the definition (yes, no or borderline). The second is the kind of file (runtime log written by the project, sample telemetry, test fixture, or non-telemetry). The third is whether a maintainer would treat the finding as a defect in telemetry the project emits. The labels and a one-line rationale for each are in `results/rq3/validation_labels.csv`. A blinded packet and agreement script for independent human raters are in `docs/evaluation/annotation/`.

Results. Table II summarises the scan and the validation. The scan found no missing-duration or unbounded-cardinality findings. Of the 14 telemetry-bearing repositories with at least one failure event, 10 have a failure event without a recognised correlation id.

The failure-path checks matched their definitions in 25 of 30 sampled findings (Wilson 95% CI 0.66–0.93). In four of the five mismatches the evidence is in the event under a field the checks do not read. Java logback events hold `trace_id` under a nested `mdc` object (one finding) or the exception class under a nested `throwable.className` (two findings), and one test-fixture record carries the OpenTelemetry attribute `error.type`, a name missing from the error-evidence list. Descending into nested objects resolves the `mdc` case. The other three also need `className` and `error.type` added to the error-evidence field list. The fifth is a disaster-recovery drill specification parsed as a failure event because of its category name.

The privacy checks matched their definitions in 9 of 15 sensitive-value findings and 1 of 15 unclassified-identifier findings, with nine more `session_id` fields judged borderline. Most mismatches come from datasets stored as JSON, where the password and bearer patterns match prose or code text, and from the `token` name heuristic matching token counts.

The file-kind labels separate project telemetry from demo data. Of the 60 sampled findings, 21 came from runtime logs the project wrote, 22 from demo telemetry, 5 from test fixtures and 12 from other JSON such as datasets. Seven were defects in the project’s own telemetry. Four are committed error-log entries with no exception type or error code, two are failures without request ids, and one is a committed agent prompt log

containing a personal email address.

Answer. On public repositories the failure-path checks match their definitions in 25 of 30 sampled findings. In four of the five mismatches the evidence is present under a nested object or an unrecognised field name.

D. Boundary controls

A prospectively recorded local audit exercised nine hand-constructed evidence cases, six trace-identity cases and two observational-signature cases (`audit/semantic_controls.py`). Empty containers, booleans and blank strings originally cleared both failure-path gaps; they now retain both findings. Explicitly conflicting trace identifiers originally attached a child span or log to another trace via a shared span identifier. Resolution now requires a unique matching trace/span identity, with a unique span-only fallback when the trace is absent. Correlation-key namespaces remain distinct. Two requirements for the same named signal originally collapsed in the equivalence signature; removing a required field now changes the signature even when both questions remain unsatisfied.

A fresh 18-case nested-identity packet found four exceptions when container-valued span identifiers occurred in span links or metric exemplars. Applying the same scalar validation to these nested identities removed all four exceptions while retaining valid associations and rejecting conflicting traces. All 18 constructed cases passed after repair (`audit/nested_identity.py`). A fresh independent reviewer then found 28 false associations when explicit malformed trace identifiers were normalized to missing identity, plus four scalar-collection exceptions. The repair now preserves the distinction between absent and explicitly invalid trace metadata and requires list-valued links and exemplars. These additional attacks show why the earlier 18 passing cases were insufficient. These controls test implementation boundaries, not general diagnostic accuracy. Recognized error/status values need not identify the actual cause, and recognized identifiers need not join the whole failed request. A separate frozen control added synthetic placeholder fields to two copied failure events. Its coverage score rose from 45 to 100 although no actual request or cause facts were added. This score measures the field projection; no intervention on real instrumentation or improvement in incident resolution is established. Finite model witnesses are executable checks, not universal proofs.

VI. THREATS TO VALIDITY

Labels. The gold-set labels were written by the tool’s author, who also wrote the hard items, and the RQ3 labels come from a single LLM rater. The released packet (`docs/evaluation/annotation/`) and agreement script (`scripts/annotation_agreement.py`) support an independent human rating of both. The LLM judges in RQ2 and the LLM rater in RQ3 may share failure modes, although the two tasks use different inputs and questions.

Sample sizes. With 15 findings per class, the intervals in Table II are 0.34 to 0.44 wide. The McNemar tests in RQ2 have little power at $n = 112$ with few discordant pairs.

LLM baseline. We used one zero-shot prompt per model with no tuning, and a single recorded sample per item. The committed log is the record of this run.

Corpus. The corpus was selected for parseable telemetry rather than sampled at random, five of its subjects were chosen by hand, and Python projects dominate (34 of 58). Prevalence figures describe this corpus. The local audit recounted the archived records; source checkouts were unavailable, so it did not re-harvest all 58 repositories with the repaired checker. The original 8.5-minute scan is not a fresh timing measurement.

VII. CONCLUSION

TELEMETRY-CONTRACTS checks failure-path telemetry in data projects already emit, without configuration. On the gold set its F1 of 0.953 falls between those of two zero-shot LLM judges (0.991 and 0.952; neither difference is significant at this sample size), and it runs offline and deterministically at no per-event cost. On 58 public repositories its correlation and error-evidence checks matched their definitions in 25 of 30 LLM-labelled findings. The gold set, corpus manifest, raw model outputs and annotation packet are released for reuse.

ARTIFACT AVAILABILITY

Source, gold set, corpus manifest, LLM prompts and raw outputs, corpus findings and validation labels: <https://github.com/thehalleyyoung/telemetry-contract-semantics>. See `REPRODUCE.md`.

REFERENCES

- [1] D. Yuan, S. Park, and Y. Zhou, “Characterizing logging practices in open-source software,” in *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, 2012, pp. 102–112.
- [2] B. Chen and Z. M. J. Jiang, “Characterizing logging practices in Java-based open source software projects – a replication study in Apache Software Foundation,” *Empirical Software Engineering*, vol. 22, no. 1, pp. 330–374, 2017.
- [3] Y. Zeng, J. Chen, W. Shang, and T.-H. P. Chen, “Studying the characteristics of logging practices in mobile apps: A case study on F-Droid,” *Empirical Software Engineering*, vol. 24, no. 6, pp. 3394–3434, 2019.
- [4] D. Yuan, J. Zheng, S. Park, Y. Zhou, and S. Savage, “Improving software diagnosability via log enhancement,” in *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2011, pp. 3–14.
- [5] D. Yuan, S. Park, P. Huang, Y. Liu, M. M. Lee, X. Tang, Y. Zhou, and S. Savage, “Be conservative: Enhancing failure diagnosis with proactive logging,” in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2012, pp. 293–306.
- [6] Q. Fu, J. Zhu, W. Hu, J.-G. Lou, R. Ding, Q. Lin, D. Zhang, and T. Xie, “Where do developers log? An empirical study on logging practices in industry,” in *Companion Proceedings of the 36th International Conference on Software Engineering (ICSE Companion)*, 2014, pp. 24–33.
- [7] J. Zhu, P. He, Q. Fu, H. Zhang, M. R. Lyu, and D. Zhang, “Learning to log: Helping developers make informed logging decisions,” in *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2015, pp. 415–425.
- [8] Z. Li, T.-H. P. Chen, and W. Shang, “Where shall we log? Studying and suggesting logging locations in code blocks,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2020, pp. 361–372.

- [9] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspán, and C. Shanbhag, “Dapper, a large-scale distributed systems tracing infrastructure,” Google, Inc., Tech. Rep. dapper-2010-1, 2010.
- [10] OpenTelemetry Authors, “OpenTelemetry semantic conventions,” <https://opentelemetry.io/docs/specs/semconv/>, accessed 2026-10-06.
- [11] H. Li, W. Shang, B. Adams, M. Sayagh, and A. E. Hassan, “A qualitative study of the benefits and costs of logging from developers’ perspectives,” *IEEE Transactions on Software Engineering*, vol. 47, no. 12, pp. 2858–2873, 2021.
- [12] J. Xu, Z. Cui, Y. Zhao, X. Zhang, S. He, P. He, L. Li, Y. Kang, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, “UniLog: Automatic logging via LLM and in-context learning,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024, pp. 14:1–14:12.
- [13] Y. Li, Y. Huo, Z. Jiang, R. Zhong, P. He, Y. Su, L. Briand, and M. R. Lyu, “Exploring the effectiveness of LLMs in automated logging generation: An empirical study,” arXiv:2307.05950, 2023.
- [14] B. Chen and Z. M. J. Jiang, “Characterizing and detecting anti-patterns in the logging code,” in *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2017, pp. 71–81.
- [15] D. Yuan, H. Mai, W. Xiong, L. Tan, Y. Zhou, and S. Pasupathy, “SherLog: Error diagnosis by connecting clues from run-time logs,” in *Proceedings of the 15th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2010, pp. 143–154.
- [16] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, “A survey on automated log analysis for reliability engineering,” *ACM Computing Surveys*, vol. 54, no. 6, pp. 130:1–130:37, 2022.
- [17] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, “Tools and benchmarks for automated log parsing,” in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 121–130.
- [18] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, “Loghub: A large collection of system log datasets for AI-driven log analytics,” in *Proceedings of the 34th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, 2023, pp. 355–366.
- [19] G. Ammons, R. Bodík, and J. R. Larus, “Mining specifications,” in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2002, pp. 4–16.
- [20] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, “The Daikon system for dynamic detection of likely invariants,” *Science of Computer Programming*, vol. 69, no. 1–3, pp. 35–45, 2007.
- [21] C. Le Goues and W. Weimer, “Specification mining with few false positives,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. LNCS, vol. 5505, 2009, pp. 292–306.
- [22] M. Leucker and C. Schallhart, “A brief account of runtime verification,” *The Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.
- [23] A. Bauer, M. Leucker, and C. Schallhart, “Runtime verification for LTL and TLTL,” *ACM Transactions on Software Engineering and Methodology*, vol. 20, no. 4, pp. 14:1–14:64, 2011.
- [24] M. R. Clarkson and F. B. Schneider, “Hyperproperties,” *Journal of Computer Security*, vol. 18, no. 6, pp. 1157–1210, 2010.